

Diagramme d'objets

Définition

- Un diagramme d'objets est une instance d'un diagramme de classes illustrant l'état d'un système à un moment donné. Il s'agit d'une représentation d'un ensemble d'objets et de liens, exprimant sa structure statique.
- Un diagramme d'objets permet de valider un diagramme de classes et de faciliter la compréhension des structures de données complexes.

Formalisme

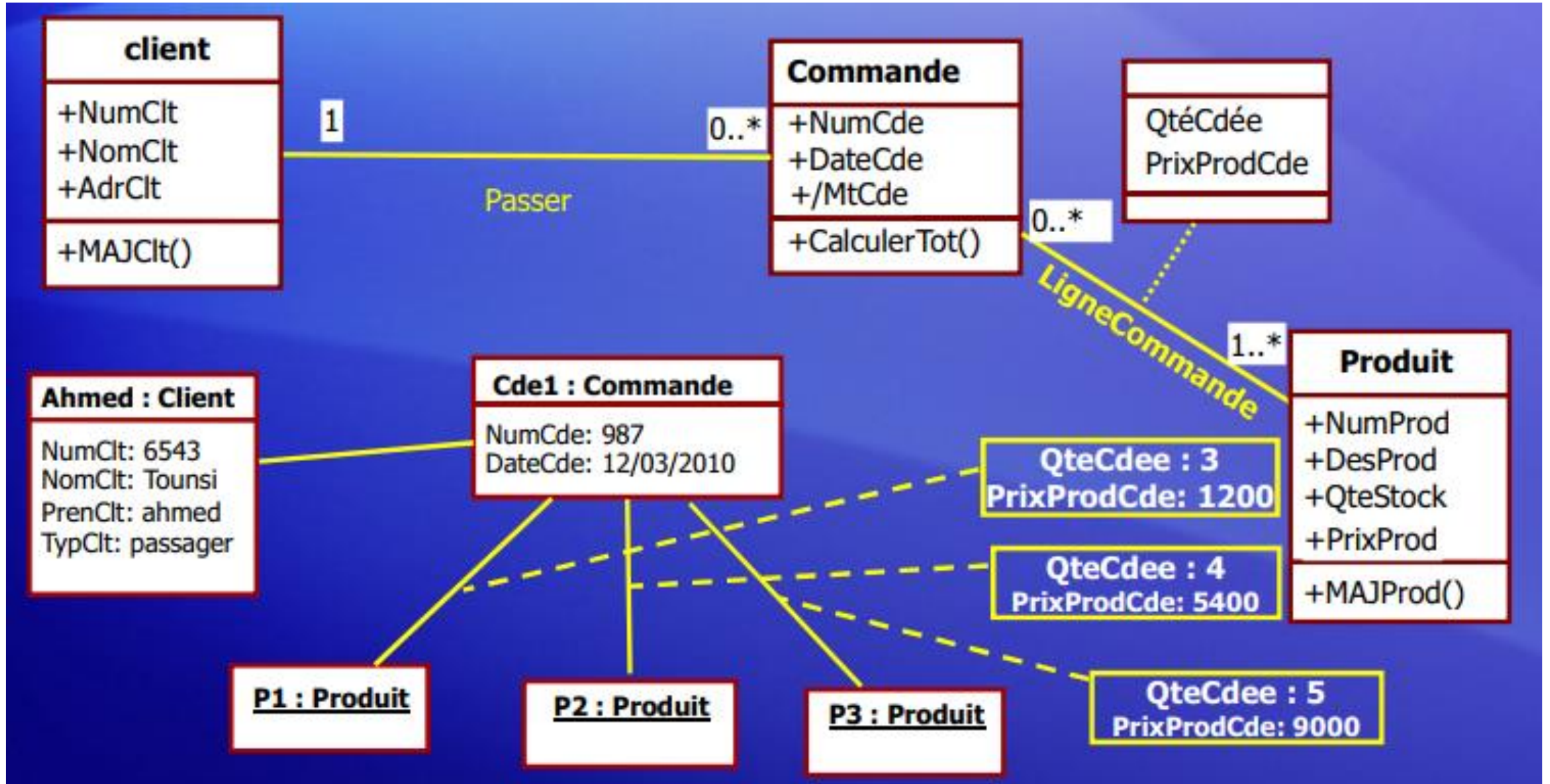
Un objet est une instance d'une classe : il représente l'état d'une classe à un instant précis.

Ahmed:Client
NumCl : 6543 NomCl : mohamed PrenCl : ahmed TypCl : passager

Objet comportant les valeurs
des attributs de sa classe

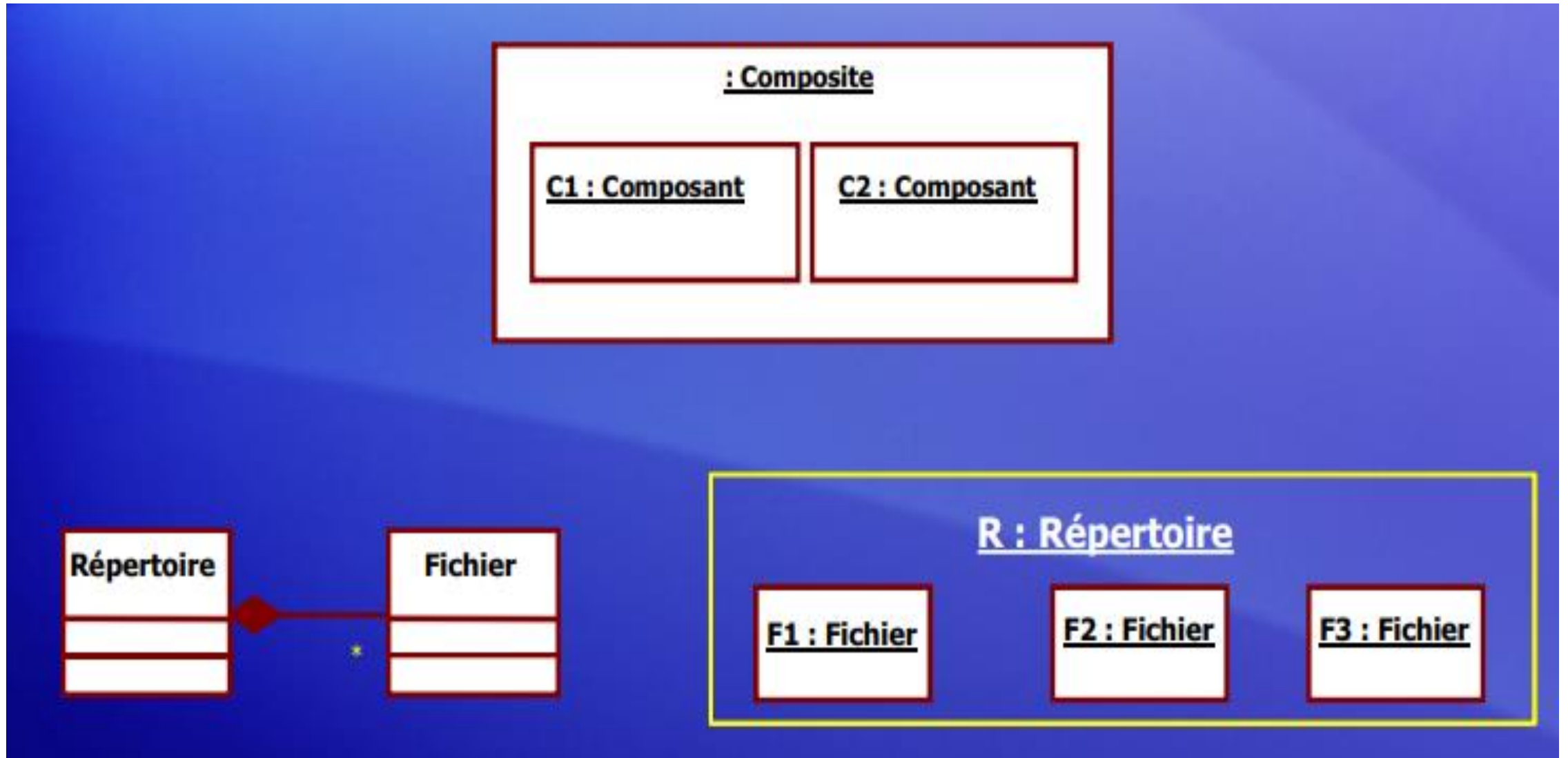
Exemple

Cas d'association



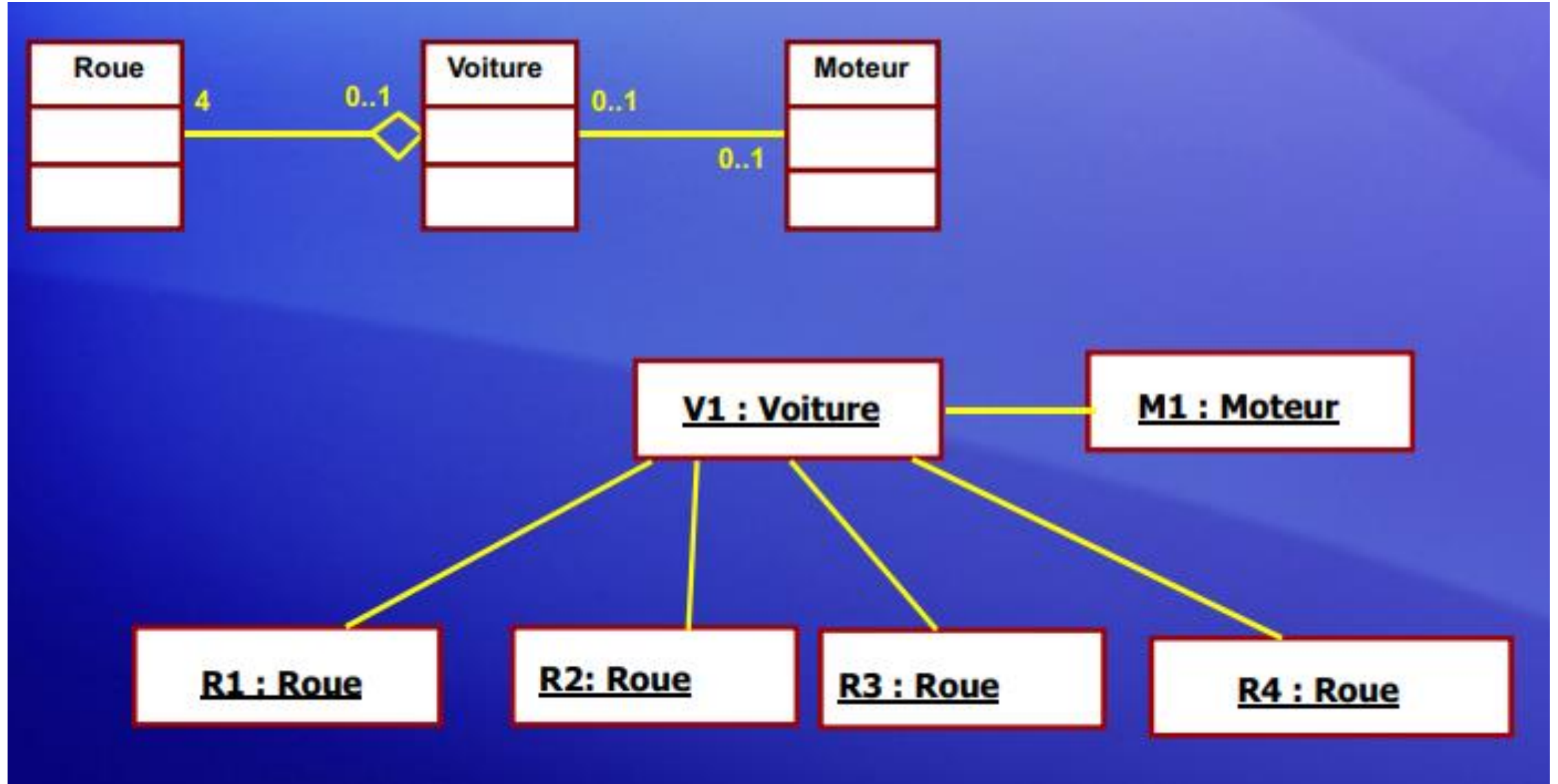
Exemple

Cas de composition



Exemple

Cas d'agrégation



Transformation du diagramme de classes en un schéma relationnel

Transformation des éléments d'une classe

Règle1 : Transformation d'une classe

- Chaque classe se transforme en une **relation** portant le nom de la classe et comportant ses attributs.
- Pour la définition de sa **clé primaire**, il s'agit de trouver un ou plusieurs attributs permettant d'identifier tout tuple de la relation d'une façon unique.
- Si aucun attribut n'est trouvé, il s'agit d'en ajouter un, jouant ainsi le rôle de clé primaire.

Transformation des éléments d'une classe

Règle1 : Transformation d'une classe (exemple)



Transformation des éléments d'une classe

Règle 2 : Transformation des attributs multivalués

- Pour chaque attribut multivalué b d'une classe C :
 - (i) on crée une nouvelle relation RB , qui comprend un attribut monovalué correspondant à b , plus la clé de la relation représentant C ;
 - (ii) la clé de RB est la concaténation des deux attributs.
 - La clé de la relation RB n'est valide que si les valeurs de b sont uniques pour toute instance de C . Sinon, il faut ajouter un attribut numSéquentiel à RB et la clé sera : la clé de C concaténée à numSéquentiel.

Transformation des associations

Règle 2 : Transformation des attributs multivalués (exemple)



Transformation des associations

Règle 3 : Cas d'une association binaire 1 - N

Toute association 1-N se traduit par l'ajout d'une clé étrangère dans la relation correspondante à la classe de multiplicité 1, faisant référence à la clé primaire de la relation correspondante à la classe de multiplicité N.



Transformation des associations

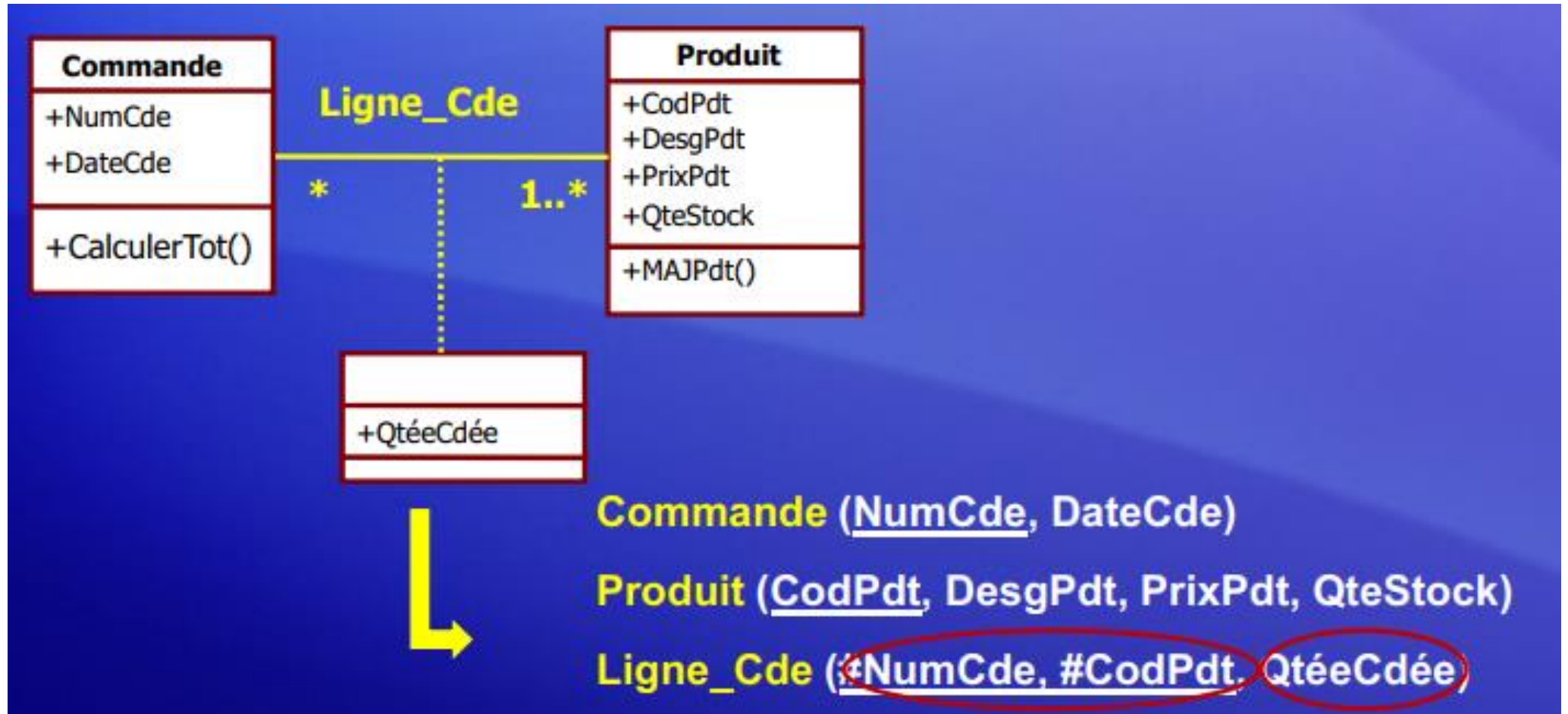
Règle 4 : Cas d'une association binaire M..N

- Toute association M..N se traduit en une relation ayant comme clé la concaténation des clés des relations correspondantes aux classes intervenantes.
- Dans le cas d'une association attribuée / classe association possédant des attributs, la relation résultante comportera en plus ces derniers attributs.



Transformation des associations

Règle 4 : Cas d'une association binaire M..N



Transformation des associations

Cas d'une association réflexive

Une relation réflexive se traite de la même manière qu'une association, en tenant en compte les multiplicités de ses extrémités.



Transformation des associations

Cas d'une association réflexive

Vu l'ambiguïté de compréhension de la relation « Parent », il s'avère nécessaire d'ajouter un attribut rôle distinguant le père de la mère.



Transformation des associations

Cas de composition

Une relation de composition est transformée de la même manière qu'une association 1..N (Règle 3).

- La clé primaire du composant est la concaténation de sa clé avec la clé étrangère issue de la classe composée.

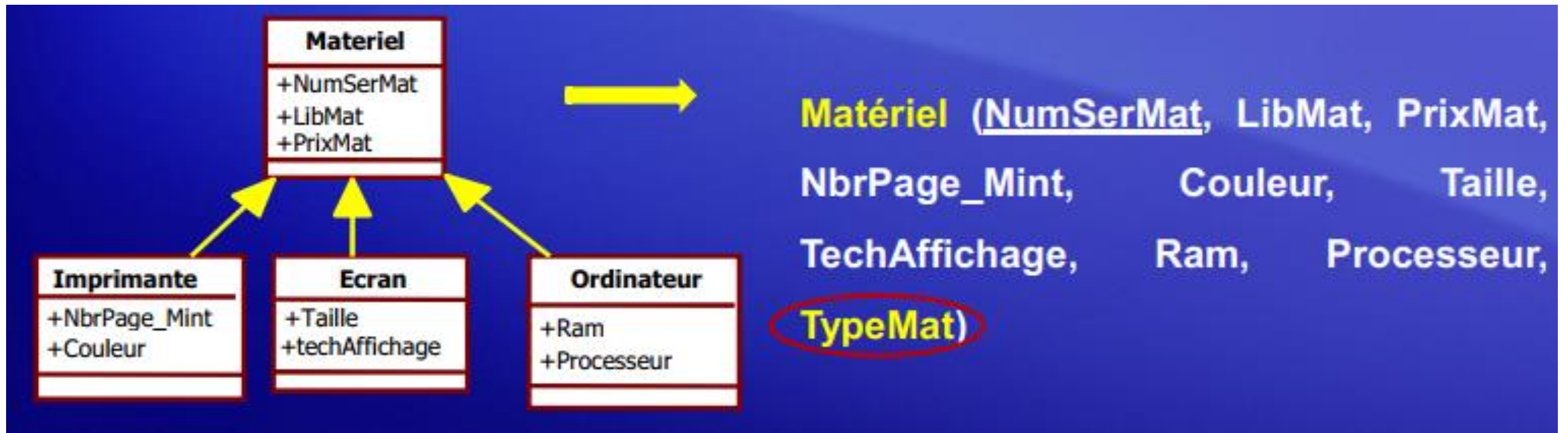


Transformation des associations

Généralisation/Spécialisation

Règle 6 : Décomposition ascendante

- Il s'agit de créer une relation correspondante à la classe mère et contenant en plus tous les attributs des classes-filles. Puis, y ajouter un attribut pour distinguer les types des objets.

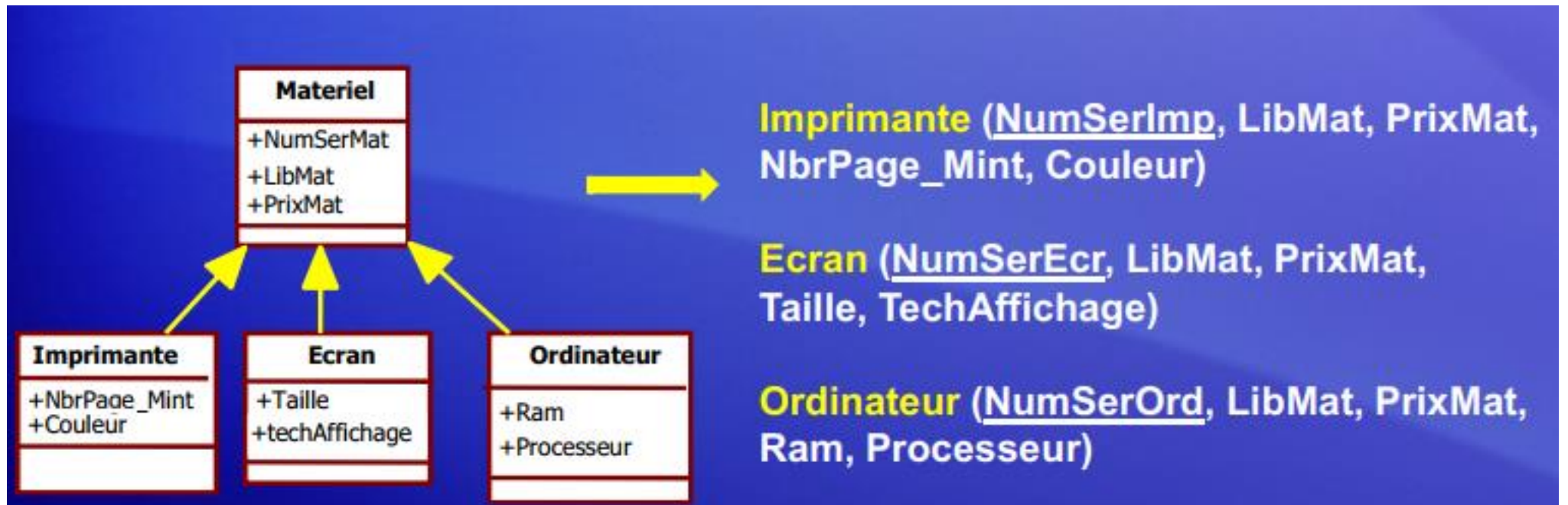


Transformation des associations

Généralisation/Spécialisation

Règle 7 : Décomposition descendante

- Cette règle est généralement adoptée lorsqu'il existe une contrainte {complet} ou {partition} sur la généralisation.
- La classe-mère n'est pas traduite en une relation et seules les sous-classes qui le sont.
- Chacune des sous-classes contient en plus de ses attributs spécifiques ceux de la classe-mère.

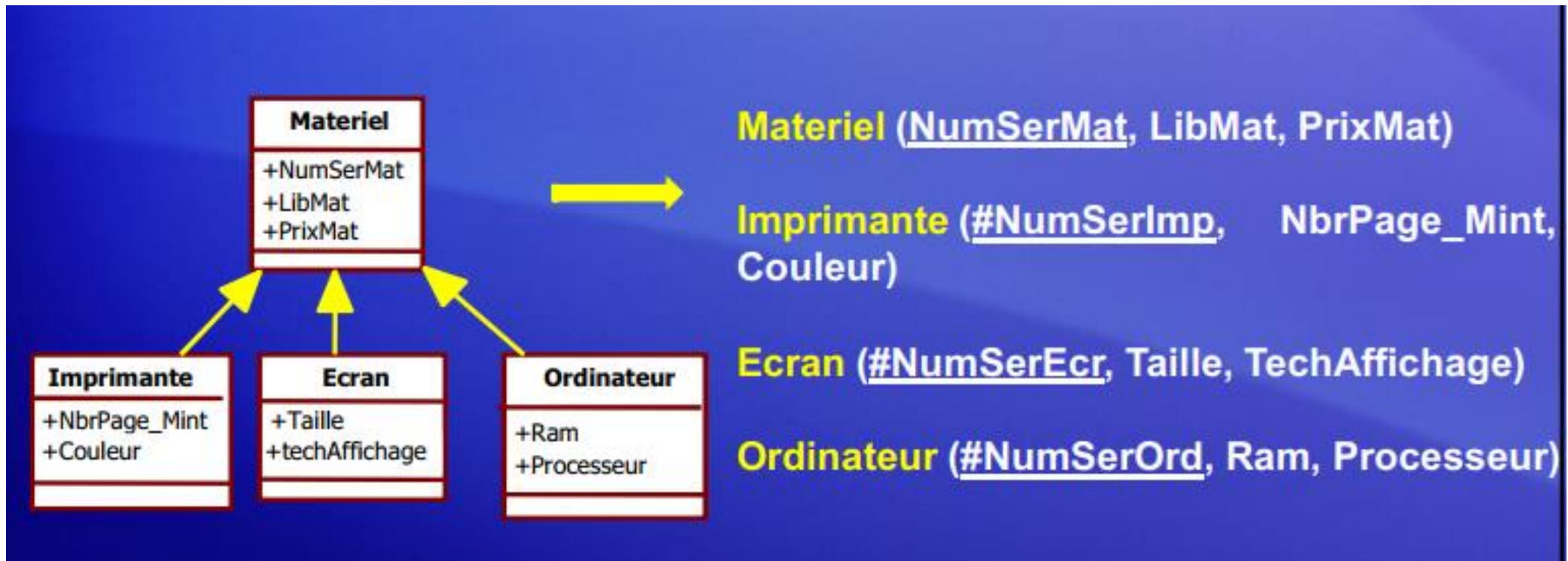


Transformation des associations

Généralisation/Spécialisation

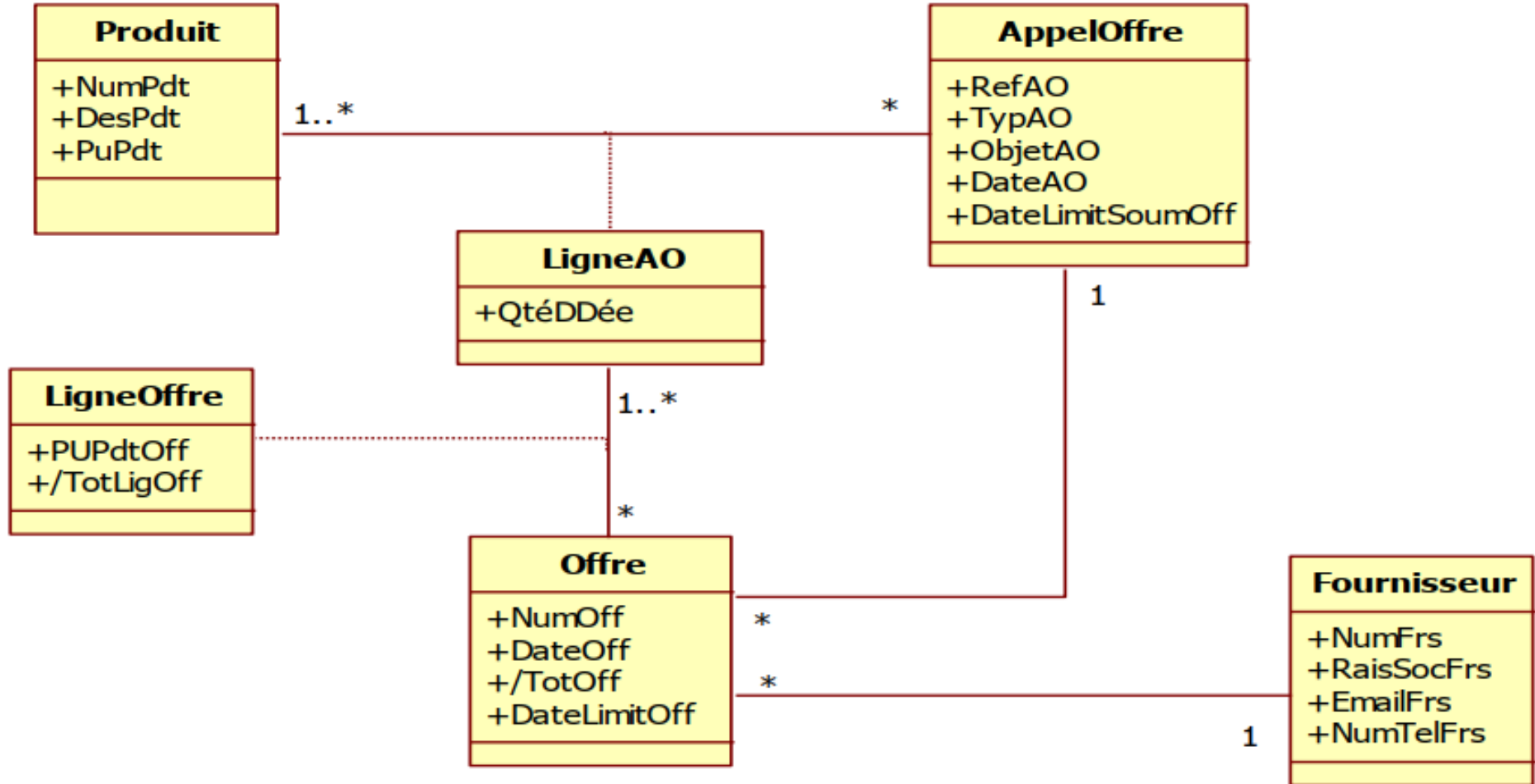
Règle 8 : Décomposition par distinction

- La super-classe ainsi que les sous-classes sont transformées en relations.
- La clé primaire de la relation issue de la classe-mère migre dans la ou les relations issues des sous-classes et devient à la fois clé primaire et clé étrangère



Exercice

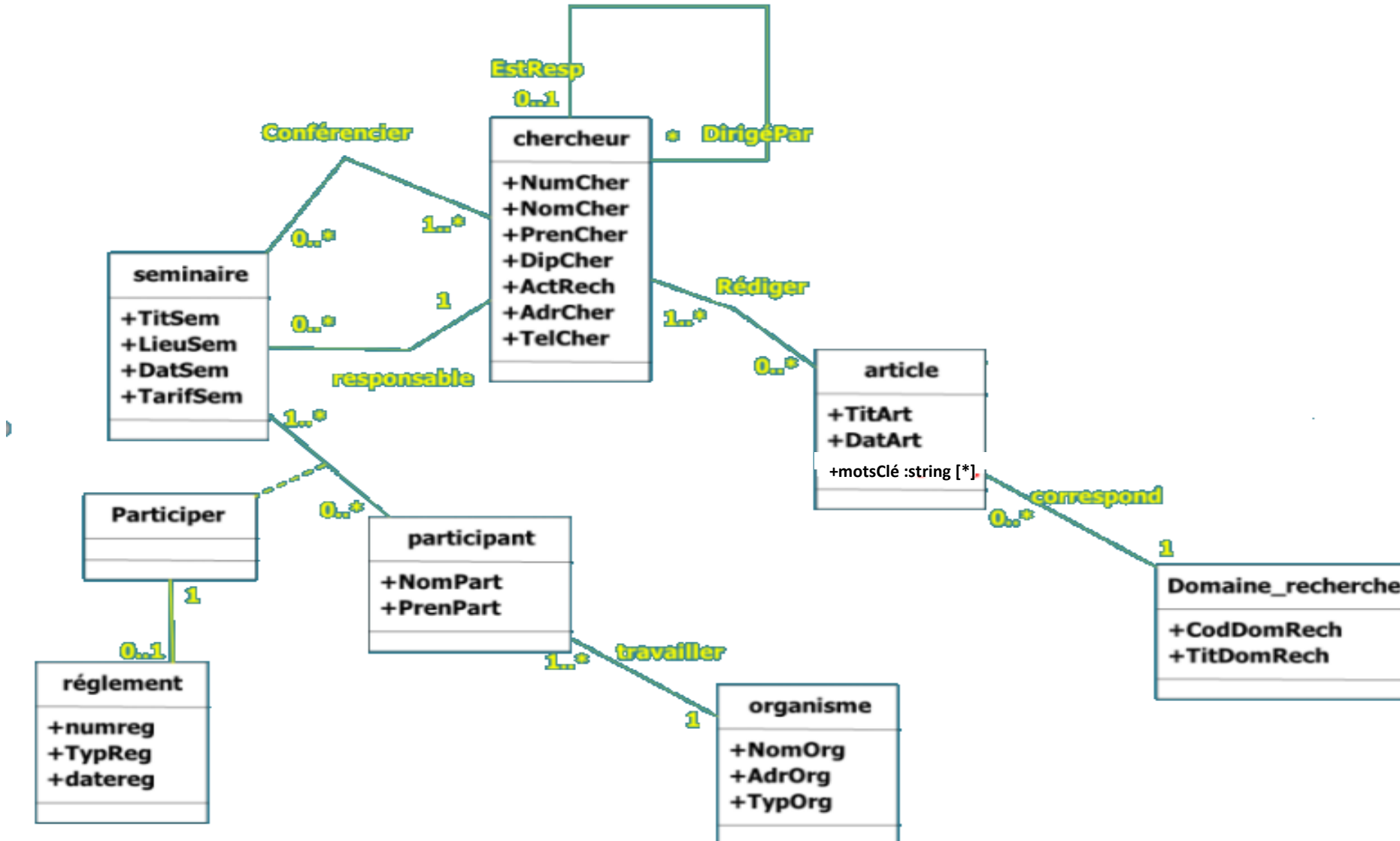
Considérons le diagramme de classes suivant, relatif à la gestion des Appels d'Offres (AO) d'une société et des offres proposées par ses fournisseurs :



Elaborez le schéma relationnel correspondant au diagramme de classes ci-dessus.

Exercice

Considérons le diagramme de classes suivant, relatif à la gestion des Appels d'Offres (AO) d'une société et des offres proposées par ses fournisseurs :



Élaborez le schéma relationnel correspondant au diagramme de classes ci-dessus.

Concepts Fondamentaux de la COO

Concepts fondamentaux de la COO

- Abstraction
- Encapsulation
- Modularité
- Hiérarchie (Héritage)
- Polymorphisme

Concepts fondamentaux de la COO

Abstraction

- L'abstraction consiste à :
 - modéliser l'essentiel
 - cacher les détails d'implémentation
- On répond à :
 - *Que fait l'objet ?*
 - sans expliquer *comment il le fait*

➔ Simplifier la complexité du système

Exemple réel:



Voiture

- Tu utilises : volant, pédales
- Tu ignores : moteur, mécanique interne

◆ En programmation

- **Classe abstraite** → modèle général
- **Interface** → comportement attendu

Concepts fondamentaux de la COO

Abstraction

```
interface Animal {  
void faireSon(); // méthode abstraite par défaut dans une  
interface  
}
```

👉 Animal = abstraction

```
class Chien implements Animal {  
@Override  
public void faireSon() {  
System.out.println("Aboyer");  
}  
}
```

👉 Chien = implémentation concrète

- L'implémentation de la classe est cachée pour l'utilisateur: Définition d'une interface
- Possibilité de modifier l'implémentation sans modifier l'interface
- Facilité de l'évolution de l'objet/maintenance

Concepts fondamentaux de la COO

Encapsulation

- Mécanisme consistant à rassembler, au sein d'une même structure, les données et les Traitements: Définition des attributs et méthodes au niveau de la classe
- Préservation de l'intégrité des données: l'accès direct aux attributs est interdit et l'interaction entre les objets se fait uniquement grâce aux méthodes (souvent appelées accesseurs: getters et setters)

Imaginons une **classe conceptuelle** CompteBancaire :

- **Attributs encapsulés** : solde, propriétaire
- **Méthodes d'accès** : getSolde(), deposer(montant), retirer(montant)

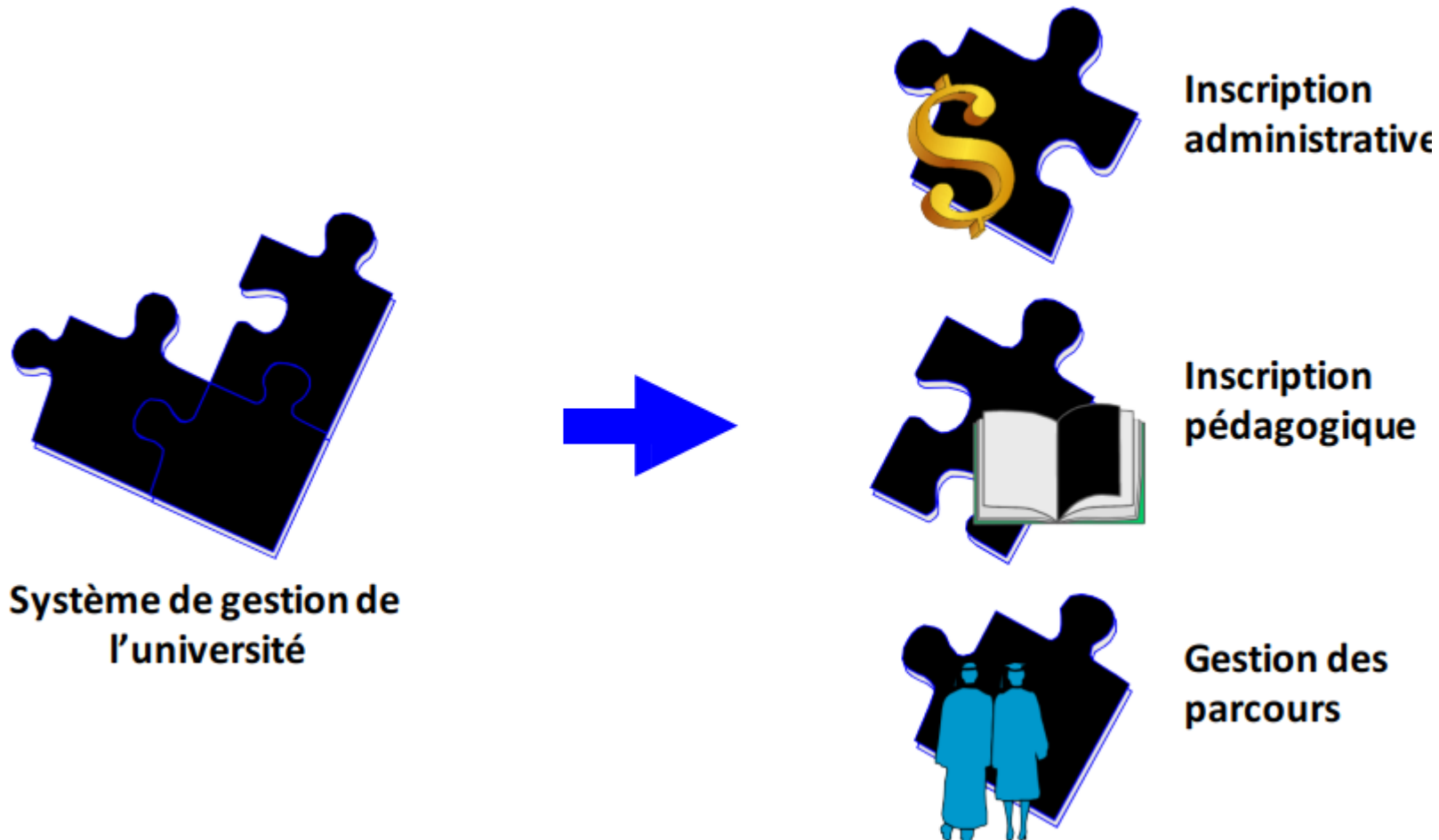
```
Classe : CompteBancaire
-----
- solde : réel (privé)
- propriétaire : chaîne (privé)

+ getSolde() : réel
+ deposer(montant : réel) : vide
+ retirer(montant : réel) : vide
```

Concepts fondamentaux de la COO

Modularité

- Casser un système en sous modules



Concepts fondamentaux de la COO

Héritage

- Un objet spécialisé bénéficie ou hérite des caractéristiques de l'objet le plus général, auquel il rajoute ses éléments propres
- Création de nouvelles classes basées sur des classes existantes
 - Transmission des propriétés (attributs et méthodes) de la classe mère vers la classe fille
 - Traduit la relation « est un ... »
- Deux orientations possibles
 - Spécialisation : Ajout / adaptation des caractéristiques
 - Généralisation : Regroupement des caractéristiques communes
- Possibilité d'héritage multiple
 - Avantages
 - Éviter la duplication du code
 - Encourager la réutilisation du code

Concepts fondamentaux de la COO

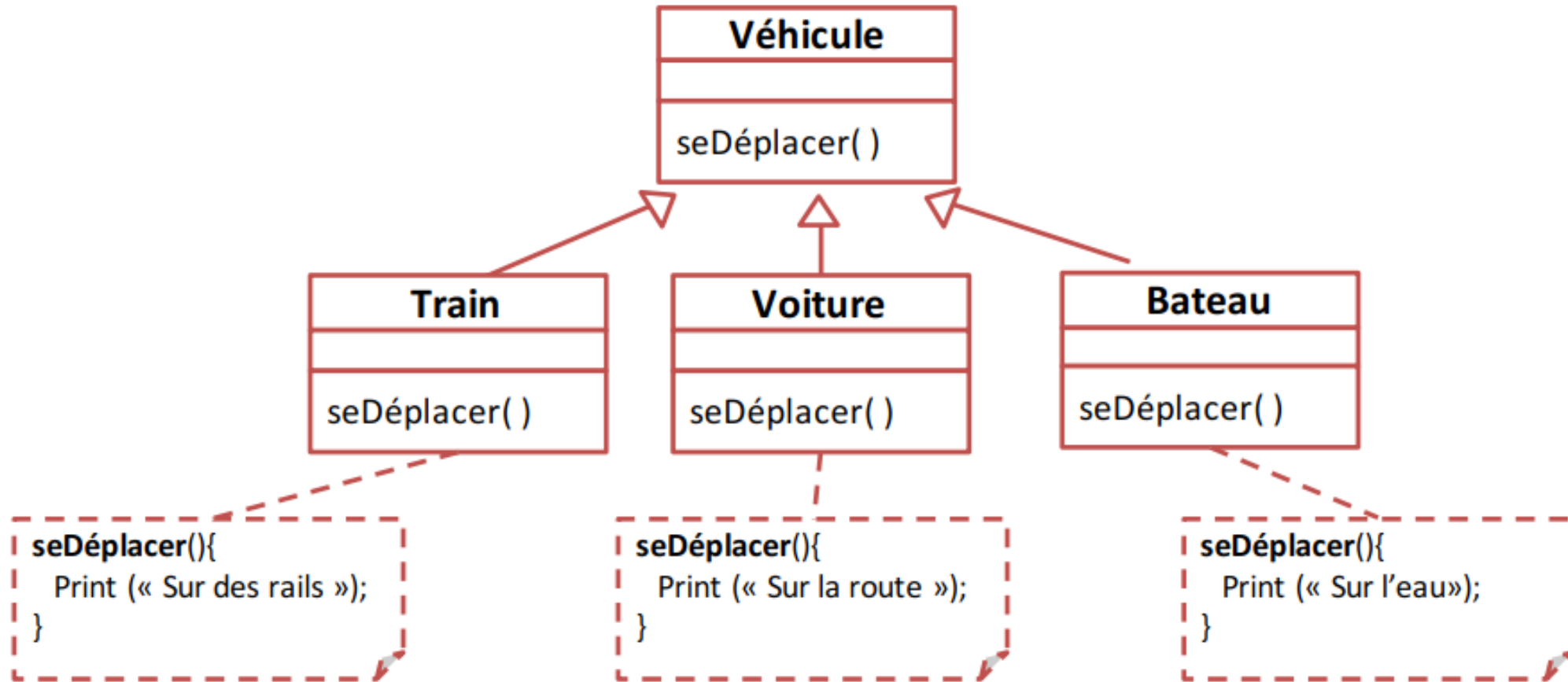
Polymorphisme

Poly : plusieurs ***Morphisme*** : Forme

- Faculté d'une méthode à pouvoir s'appliquer à des objets de classes différentes
- Capacité d'une classe à redéfinir une méthode héritée à partir d'une classe mère
 - Surcharge

Concepts fondamentaux de la COO

Polymorphisme



Concepts fondamentaux de la COO

Exercice

- Une entreprise a des **employés** :
- Tous les employés ont **nom**, **ID** et **salaire**.
- Chaque type d'employé a sa façon de calculer le salaire :
 - Employé fixe : salaire mensuel
 - Employé horaire : nombre d'heures $\times$ taux horaire

1- Elaborer le diagramme de classe

2- Implémenter le code en java